The Find dialog's layouts

One subtle aspect of the layout manager classes is that they are not widgets. Instead, they are derived from QLayout, which in turn is derived from QObject. In the figure, widgets are represented by solid outlines and layouts are represented by dashed outlines to highlight the difference between them. In a running application, layouts are invisible.

When the sublayouts are added to the parent layout (lines 25, 33, and 34), the sublayouts are automatically reparented. Then, when the main layout is installed on the dialog (line 35), it becomes a child of the dialog, and all the widgets in the layouts are reparented to become children of the dialog.

```
36     setWindowTitle(tr("Find"));
37     setFixedHeight(sizeHint().height());
38 }
```

Finally, we set the title to be shown in the dialog's title bar and we set the window to have a fixed height, since there aren't any widgets in the dialog that can meaningfully occupy any extra vertical space. The QWidget::sizeHint() function returns a widget's "ideal" size.

This completes the review of FindDialog's constructor. Since we used new to create the dialog's widgets and layouts, it would seem that we need to write a destructor that calls delete on each widget and layout we created. But this isn't necessary, since Qt automatically deletes child objects when the parent is destroyed, and the child widgets and layouts are all descendants of the FindDialog.

Now we will look at the dialog's slots:

```
39 void FindDialog::findClicked()
40 {
41     QString text = lineEdit->text();
42     Qt::CaseSensitivity cs =
43             caseCheckBox->isChecked() ?
Qt::CaseSensitive
44                                       :
Qt::CaseInsensitive;
45     if (backwardCheckBox->isChecked()) {
46         emit findPrevious(text, cs);
47     } else {
48         emit findNext(text, cs);
49     }
50 }
```